MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 8

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: run, inside xv6 **3**

2 Part 2: pingpong **8**

Conclusion **12**

References **12**

Goal

In the previous laboratory work the calls fork, exec and wait were used on Linux, where they are offered by the C library of a system of millions of lines. The purpose of this laboratory work is to run the same program inside xv6, the small operating system built during the semester, and to see that the calls are the same: there is no glibc here, only the system calls of the kernel that was compiled by hand. What changes is the implementation behind the call, not the interface that the program uses.

The work has two parts. In the first part the program user/run.c performs the three steps of the shell inside xv6: it copies itself with fork, the child becomes another program with exec, and the parent collects it with wait. The child is made to become echo, then ls, and finally a program that does not exist, in order to observe what the parent sees when exec fails. In the second part the program user/pingpong.c makes two processes talk to each other through two pipes: the parent sends one byte to the child, the child answers with the same byte, and each of them prints a line with its own pid. The exercise shows that a pipe is unidirectional and that reading from a pipe blocks until data arrives, which is what orders the two messages.

The work was done in the tree ~/xv6-riscv, which was built and started with make qemu and left with Ctrl-A X. Every new program is a file user/NAME.c plus one line \$U/_NAME\ in the list UPROGS of the Makefile, written with a TAB at the beginning and with nothing after the backslash.

Part 1: run, inside xv6

Questions:

- Write the three calls that run.c uses in xv6 and the three calls run.c used on Linux in Lab 7. What is the same?
- Who prints "hello from the child": the code of run, or echo?
- Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)

Answers:

- In xv6 the program uses fork(), exec(path, argv) and wait(&status); on Linux it used fork(), execlp(file, arg0, ..., NULL) and waitpid(pid, &status, 0). The three steps are the same, in the same order and with the same meaning: one call creates a copy of the process, one call replaces the program of that copy, and one call makes the parent wait for its end and gives it the status. The differences are only in the shape of the interface: exec of xv6 takes the array of arguments directly and does not search PATH, so the name of the program is a path, and wait of xv6 waits for any child, while waitpid can wait for a chosen one. Moreover, on Linux execlp and waitpid are functions of the C library that call the kernel, while in xv6 they are the system calls themselves. The comparison is given in Table 1.

- echo prints it. The code of run prints only the two lines with the pids; after exec succeeds, the memory image of the child is replaced and the program that executes from that moment is echo, which writes its arguments hello from the child. This is also why the line printf("exec failed\n") is not printed in that case.
- Because the status is chosen by the child itself, in the line that follows exec. In xv6 that line is exit(1), so the parent sees 1; on Linux the corresponding program ended with exit(127), the value that a shell uses for "command not found", so the parent saw 127. The kernel does not invent either number: it only transmits the value given to exit, and if the two programs were written with the same constant, the two systems would show the same status.

user/run.c – the program performs the three steps of the shell inside xv6. It prints its own pid obtained with getpid, then calls fork, which returns 0 in the child and the pid of the child in the parent, so that the same code separates into two processes. The child prints its pid, builds the array of arguments, where the first element is the name of the program and the last one is 0, and calls exec; if exec returns, the call has failed, and the child prints a message and ends with exit(1). The parent calls wait(&status), which blocks it until the child ends, and then prints the status. The source is given below.

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from
    the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                        // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                            // only the child runs
   this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0
   };
    exec("echo", args);                    // step 2: become echo
    printf("exec failed\n");               // runs only if exec
   failed
    exit(1);
  }

  int status;
  wait(&status);                           // step 3: wait for the
   child
  printf("parent: child %d ended with status %d\n", pid, status)
   ;
```

```
27    exit(0);
28 }
```



Figure 1.1: The beginning of user/run.c in the editor

nano user/run.c – the program was written in the directory user of the xv6 tree, where every user program of the system lives. It includes only kernel/types.h and user/user.h, the two headers that declare the system calls, because xv6 has no standard library: there is no stdio.h and no stdlib.h to include.

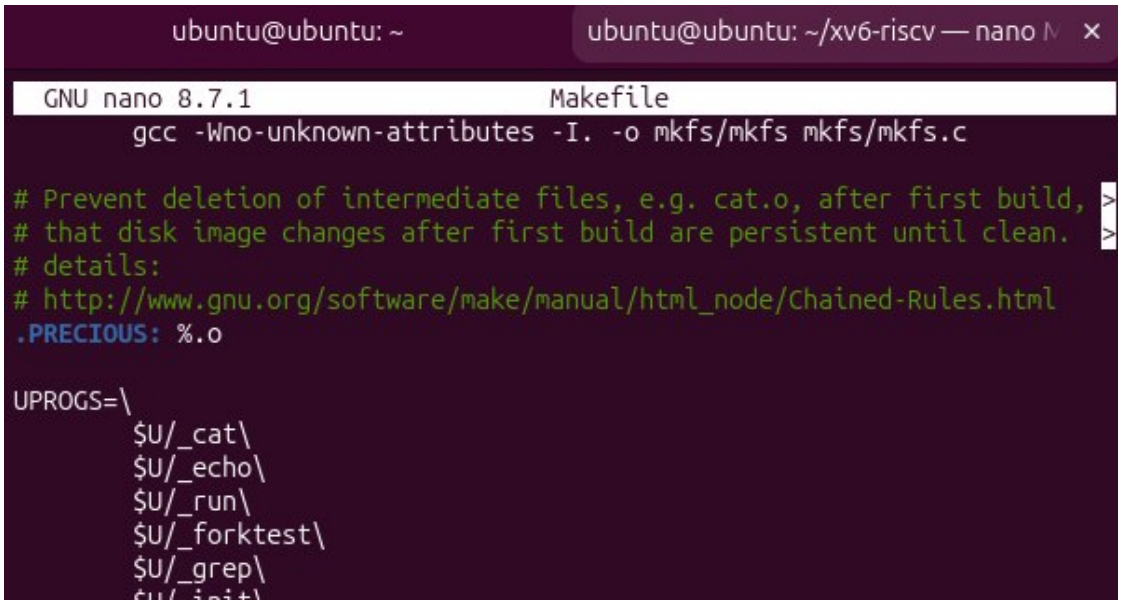


Figure 1.2: The line $U/_run\ added to UPROGS in the Makefile

Makefile – UPROGS – the lines $U/_run\ and $U/_pingpong\ were added to the variable UPROGS, each of them with a TAB at the beginning and with nothing after the

backslash, not even a space. Without these lines the programs are compiled by nothing and are not copied into the file system of xv6, so the shell answers exec run failed.



Figure 1.3: The program run inside xv6: the child becomes echo

make qemu ; $ run – xv6 is started and the program is launched from its shell. The parent has the pid 3 and the child the pid 4; the numbers start at 3 because init is the process 1 and the shell is the process 2, and xv6 never reuses a pid, since it gives them from a counter that only increases. Between the two lines of run appears hello from the child, printed by echo, and the last line, printed by the parent after wait, shows the status 0. The order of the lines is the proof of the mechanism: the parent writes first, the child and echo write in the middle, and the parent writes its last line only after the child has ended.

```
if(pid == 0){                          // only the child runs this
  printf("child: my pid is %d\n", getpid());
  char *args[] = {"ls", 0 };
  exec("ls", args);                  // step 2: become echo
  printf("exec failed\n");             // runs only if exec failed
  exit(1);
}
```

Figure 1.4: The two changed lines: the child becomes ls

char *args[] = { "ls", 0 }; exec("ls", args); – only these two lines of the child were changed. Nothing else in the program knows which command will be executed, which shows that fork and wait do not depend on the program that is launched; the array of arguments is shorter, because ls is called without arguments.

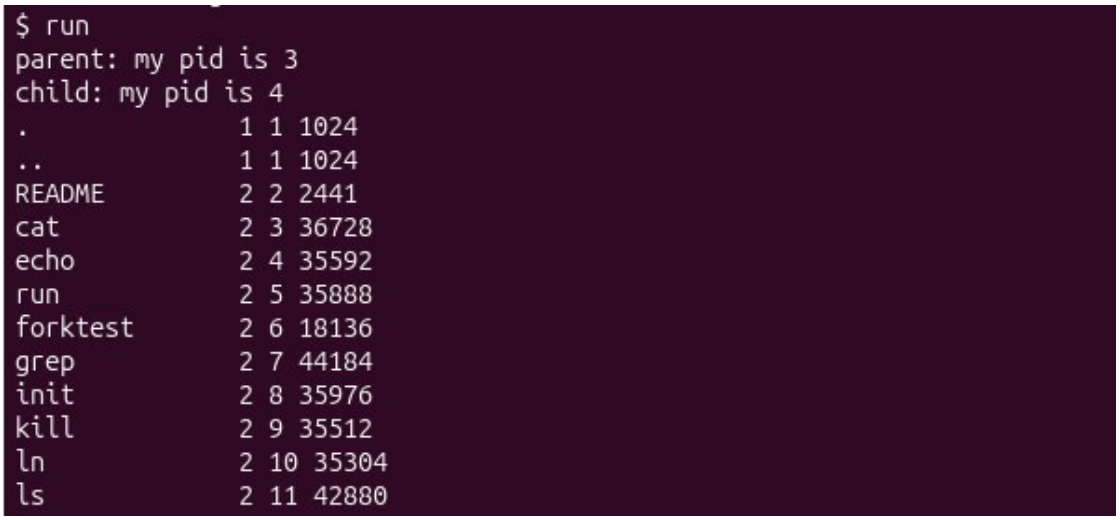


Figure 1.5: The same program with ls instead of echo

$ run (with ls) – after Ctrl-A X and a new make qemu, the program prints the same two lines with the pids, and between them appears the listing of the root directory produced by ls, with the name, the type, the inode and the size of every entry. The status is again 0 and exec failed is not printed, because the call succeeded.

```
if(pid == 0){                          // only the child runs this
  printf("child: my pid is %d\n", getpid());
  char *args[] = {"ls", 0 };
  exec("nosuchprog", args);                    // step 2: become echo
  printf("exec failed\n");             // runs only if exec failed
  exit(1);
```

Figure 1.6: The exec line asking for a program that does not exist

exec("nosuchprog", args); – the name given to exec is a program that does not exist in the file system of xv6. The child is created in the same way, but it cannot become anything else and continues to execute the code of run.



Figure 1.7: The failure of exec and the status 1

$ run (with nosuchprog) – the child prints its line, then exec returns −1 because the file is not found, the next line prints exec failed and the child ends with exit(1). The parent reports the status 1. The failure happens in the child only: the parent is intact and prints its last line normally, exactly as a shell prints an error and then shows its prompt again. Unlike Linux, xv6 has no PATH and no perror, so the name is searched

only in the current directory and the message is written by the program itself. The three executions are compared in Table 2.

Step	**In xv6**	**On Linux (Lab 7)**	**What is the same**
copy the process	fork()	fork()	identical
become another program	exec("echo", args)	execlp("ls", "ls", "-l", NULL)	replaces the image, does not return on success
wait for the child	wait(&status)	waitpid(pid, &status, 0)	blocks and gives the status

Table 1: The three calls of run.c in xv6 and on Linux

Program of the child	**Status**	**Is "exec failed" printed?**	**Reason**
echo	0	no	exec succeeded and never returned
ls	0	no	exec succeeded and never returned
nosuchprog	1	yes	exec returned -1, then exit(1)

Table 2: The three executions of run inside xv6 and the status seen by the parent

Table 1 shows that the interface of the three calls is the same in a system of millions of lines and in one of a few thousand, and Table 2 shows that the status is not produced by the kernel but chosen by the child in its own exit line.

Part 2: pingpong

Questions:

- Why do you need TWO pipes, and not one for both directions?
- Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

Answers:

- Because a pipe of xv6 is unidirectional: pipe(p) gives one end for reading, p[0], and one end for writing, p[1], and the data travel only in that direction. With a single pipe the two processes would write into the same buffer and read from the same buffer, so the parent, which reads immediately after it has written, would very probably read back its own byte instead of the answer of the child, and the roles ping and pong would collapse. Two separate pipes give two separate channels, p2c

for parent to child and c2p for child to parent, so that each byte can travel in one direction only and the answer cannot be confused with the question (Table 3).

- The call read(c2p[0], &b, 1) in the parent. Reading from a pipe that is empty blocks the process until somebody writes into it or until the last writing end is closed, so the parent stays inside read while the child prints its line and answers. This is also why "ping" is always printed before "pong", without any ordering in the code: the order is imposed by the pipe and not by the speed of the processes. The call wait(0) that follows does something else, it only collects the child that has already ended, so it does not create the order of the messages.

user/pingpong.c – the program creates the two pipes before fork, so that both processes inherit the four descriptors. After fork, each of them closes the two ends it does not use, which is necessary both for correctness and for the limited number of open files of a process. The child reads one byte from p2c[0], which blocks it until the parent writes, prints <pid>: received ping with its own pid, writes the same byte into c2p[1] and ends with exit(0). The parent writes one byte into p2c[1], then reads one byte from c2p[0], which blocks it until the answer arrives, prints <pid>: received pong and collects the child with wait(0). The completed source, with the four TODO lines replaced, is given below.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];      // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);        // the ends the child does not use
    read(p2c[0], &b, 1);                 // wait for the parent's byte
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);                // send the same byte back
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);          // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);                  // send the ping
  read(c2p[0], &b, 1);                   // blocks until the child answers
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
```

```
27    exit(0);
28 }
```

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_run\
        $U/_pingpong\
        $U/_forktest\
        $U/_grep\
        $U/_init\
        $U/_kill\
        $U/_ln\
        $U/_ls\
```

Figure 2.1: The line $U/_pingpong\ added to UPROGS in the Makefile

Makefile – $U/_pingpong – the second program needs its own line in UPROGS, written exactly like the one of run, with a TAB at the beginning and with nothing after the backslash. Without it the shell of xv6 answers exec pingpong failed, because the program was never compiled and never copied into the image of the file system.

```
ubuntu@ubuntu: ~    ubuntu@ubuntu: ~/xv6-riscv — nano u ×
 GNU nano 8.7.1                  user/pingpong.c *
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);                // wait for the parent's byte
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // send the same byte back
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
```

Figure 2.2: The beginning of user/pingpong.c with the completed lines

The four completed lines – the whole exercise consists of the two read and the two write calls; everything else was already in the given file. Each call transports exactly one byte, given by the third argument, and the second argument is the address of the variable

b, because read and write work with buffers and not with values. The byte receives the value 'x' before the first write, since an uninitialized variable would send whatever happens to be in memory, and the content of the byte is not important: only its arrival is. The close calls that were added at the end of each branch are not strictly necessary, because exit closes all the descriptors of a process, but they keep the discipline of closing every end that is no longer used.



Figure 2.3: The two processes exchanging a byte through the two pipes

make qemu ; $ pingpong – the program prints two lines: 4: received ping, written by the child, and 3: received pong, written by the parent. The pid of the ping line is the larger one, because the child was created by fork from the parent and xv6 gives the pids from a counter that only increases; the parent is the first command launched after the boot, so it has the pid 3 and its child the pid 4. If other commands are run before, or if pingpong is launched a second time, both numbers are larger, but the ping line always has the larger pid and always comes first.

Pipe	**Direction**	**Ends of the parent**	**Ends of the child**
p2c	parent → child	writes in p2c[1], closes p2c[0]	reads from p2c[0], closes p2c[1]
c2p	child → parent	reads from c2p[0], closes c2p[1]	writes in c2p[1], closes c2p[0]

Table 3: The two pipes and the ends used by each process

Step	**Parent**	**Child**	**State**
1	write(p2c[1])	blocked in read(p2c[0])	the byte arrives, the child wakes up
2	blocked in read(c2p[0])	prints received ping	the parent cannot overtake
3	blocked in read(c2p[0])	write(c2p[1]), exit(0)	the byte goes back
4	prints received pong, wait(0)	ended	the child is collected

Table 4: The order of the messages, imposed by the blocking read

Table 3 shows that each of the four ends has exactly one owner, and Table 4 shows that the alternation of the two messages is produced by the blocking of read and not by any instruction of the program.

Conclusion

The laboratory work ran inside xv6 the same program that had been run on Linux in the previous work, and showed that the interface of the operating system does not depend on its size. In the first part, user/run.c performed the three steps of the shell with fork, exec and wait: the parent received the pid 3 and the child the pid 4, the child became echo and then ls, and in both cases the status was 0 and the line exec failed was not printed, which confirms that a successful exec never returns. With a program that does not exist, exec returned, the child printed exec failed and ended with exit(1), and the parent reported the status 1; the difference from the 127 of Linux comes only from the exit line written in the two programs, and not from the kernel, since the kernel merely transmits the value given to exit. The only real differences found between the two systems are that exec of xv6 does not search a PATH, that wait waits for any child, and that there is no C library between the program and the kernel.

In the second part, user/pingpong.c made two processes exchange one byte through two pipes created before fork, each process closing the two ends it does not use. The child printed 4: received ping and the parent 3: received pong, in this order and with the larger pid first, although nothing in the code orders the two messages: the order is imposed by read, which blocks the parent until the child has answered. Two pipes were needed because a pipe is unidirectional, and with a single one the parent would have read back its own byte. Overall, the work gave a practical understanding of the birth of a process, of the meaning of the exit status and of the communication between processes through pipes, and it showed that the calls learned on Linux are the same ones that the kernel compiled during the semester implements; all the objectives of the laboratory work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *Xv6, a simple Unix-like teaching operating system: the book and the source code (MIT 6.1810)*
   `https://pdos.csail.mit.edu/6.828/xv6`
3. *Lab: Xv6 and Unix utilities (MIT 6.1810), the pingpong exercise*
   `https://pdos.csail.mit.edu/6.828/2023/labs/util.html`
4. *pipe(2) – create pipe, and the end-of-file condition of a pipe, Linux manual page*
   `https://man7.org/linux/man-pages/man2/pipe.2.html`